Common setup (run once)

```
pip install nltk
```

```
import nltk

# Download the NLTK datasets/models needed by the lab programs.
nltk.download('punkt')
nltk.download('punkt_tab')
nltk.download('brown')
nltk.download('averaged_perceptron_tagger_eng')
nltk.download('wordnet')
nltk.download('omw-1.4')
nltk.download('universal_tagset')
nltk.download('conll2000')
nltk.download('maxent_ne_chunker_tab')
nltk.download('words')
```

Run each program in Jupyter Notebook, Google Colab, or VS Code with Python installed. Some programs download corpora on first run and need an internet connection.

12. Implement a feature extractor and a robust, context-sensitive document classification model

Aim: To classify documents using extracted word features and a Naive Bayes classifier.

This example classifies movie reviews as positive or negative using NLTK's Movie Reviews corpus.

```
import nltk
from nltk.corpus import movie_reviews
from nltk.classify import NaiveBayesClassifier, accuracy

# Download the movie reviews dataset.
nltk.download('movie_reviews')

# Extract the 2,000 most frequent words from the corpus.
all_words = [
    word.lower()
    for word in movie_reviews.words()
    if word.isalpha()
]

word_frequency = nltk.FreqDist(all_words)

# Keep the most informative words as candidate features.
common_words = set(word for word, count in word_frequency.most_common(2000))

# Feature extractor: represent a document using word presence.
def extract_features(document):
    # Convert the document to a set for efficient membership checks.
    document_words = set(word.lower() for word in document)

    # Each feature indicates whether a selected word occurs in the document.
    return {
        'contains({})'.format(word): (word in document_words)
        for word in common_words
    }

# Extract features from each review and retain its category label.
documents = []

for file_id in movie_reviews.fileids():
    words = movie_reviews.words(file_id)
    label = movie_reviews.categories(file_id)[0]

    documents.append((extract_features(words), label))

# Shuffle the documents to avoid ordering bias.
import random
random.seed(42)
random.shuffle(documents)

# Split the dataset into training and testing portions.
split_point = int(0.8 * len(documents))
train_set = documents[:split_point]
test_set = documents[split_point:]

# Train the Naive Bayes document classifier.
```

```
classifier = NaiveBayesClassifier.train(train_set)

# Evaluate classification accuracy on unseen reviews.
print("Training documents:", len(train_set))
print("Testing documents:", len(test_set))
print("Accuracy:", accuracy(classifier, test_set))

# Display the most informative features.
print("\nMost informative features:")
classifier.show_most_informative_features(10)

# Classify a new review.
new_review = "The movie was excellent, enjoyable, and entertaining."
new_features = extract_features(new_review.split())

print("\nPredicted sentiment:", classifier.classify(new_features))
```

Sample output

```
Training documents: 1600
Testing documents: 400
Accuracy: [calculated accuracy]

Most informative features:
Most Informative Features
...

Predicted sentiment: pos
```

The exact accuracy and informative features depend on the training data and NLTK version.

Explanation:

1. **Feature extraction:** Converts each review into a dictionary of word-presence features.
2. **Training:** Learns relationships between features and sentiment labels.
3. **Testing:** Evaluates the classifier on reviews it did not train on.
4. **Prediction:** Assigns `pos` or `neg` to a new review.

The model uses word-presence features rather than word order, so it does not fully understand context or negation. More advanced models can include bigrams, TF-IDF, sentiment lexicons, or transformer embeddings.